Problem A. A Name of One's Own

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	256 megabytes

A new social network is launching, where n people sign up one by one. The i-th person has two usernames in mind, x_i and y_i ($x_i \neq y_i$), and must register under exactly one of them. No two people may share a username.

For each k from 1 to n, please count, modulo 998244353, the number of ways to give each of the first k people one of their two chosen usernames so that all k chosen usernames are distinct.

Input

The first line contains a single integer t ($1 \leq t \leq 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \leq n \leq 3 \cdot 10^5$) — the number of people.

The i-th of the next n lines contains two integers x_i and y_i ($1 \leq x_i, y_i \leq 10^9$, $x_i \neq y_i$) — the two usernames person i has in mind.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print n space-separated integers; the k-th of them is the number of valid username assignments for the first k people, taken modulo 998244353.

Example

standard input	standard output
4 1 1 2 2 1 2 3 4 8 7 1 7 3 7 9 2 5 5 8 2 8 8 6 10 1000000000 3 1 2 2 3 1 3	2 2 4 2 3 4 8 12 8 8 16 2 3 2

Note

In the first test case, the single person may sign up as 1 or 2, so there are 2 ways.

In the fourth test case, with only the first person there are 2 ways. With the first two people, every combination works except both taking 2, so there are 3 ways. With all three people, out of the 8 combinations only $(1, 2, 3)$ and $(2, 3, 1)$ leave no two people sharing a username, so there are 2 ways.

Problem B. Singularity

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given an array $a_1, a_2, \ldots, a_n$ of positive integers.

You can perform the following operation any number of times:

- Select a subarray $a_l, a_{l+1}, \ldots, a_r$ $(1 \le l \le r \le n)$, let k be the number of distinct elements among $a_l, a_{l+1}, \ldots, a_r$, and replace every element of that subarray by k (that is, set $a_l = a_{l+1} = \cdots = a_r = k$).

Find the minimum number of operations needed to make every element of the array equal to 1.

Input

The first line contains a single integer t $(1 \le t \le 10^4)$ — the number of test cases.

The first line of each test case contains a single integer n $(1 \le n \le 3 \cdot 10^5)$ — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ $(1 \le a_i \le 10^9)$ — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, output a single integer — the minimum number of operations needed to make all elements equal to 1.

Example

standard input	standard output
2 7 1 1 3 1 2 2 1 1 1	2 0

Note

In the first test case,

- Select the subarray $[3]$ (1 distinct value), the array becomes $[1, 1, 1, 1, 2, 2, 1]$;
- Select the subarray $[2, 2]$ (1 distinct value), the array becomes $[1, 1, 1, 1, 1, 1, 1]$.

So you can make all elements equal to 1 in 2 operations.

Problem C. Level Up

Input file:	`standard input`
Output file:	`standard output`
Time limit:	4 seconds
Memory limit:	256 megabytes

You are given an array a of n positive integers.

Consider partitioning the array into exactly k non-empty contiguous segments, such that each element belongs to exactly one segment.

For a segment $[l, r]$, its *contribution* is the number of indices i ($l \le i \le r$) such that $a_i > a_j$ for all $l \le j < i$. In other words, an element contributes if it is strictly greater than every previous element in the same segment.

For example, the contribution of the segment $[3, 1, 4, 2, 5]$ is 3; the contributing elements are 3, 4, and 5.

Your task is to divide the array into exactly k non-empty contiguous segments such that the sum of contributions of all segments is maximized.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n and k ($1 \le n \le 10^6$, $1 \le k \le \min(n, 100)$) — the length of the array and the number of segments.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed 10^6.

Output

For each test case, print a single integer — the maximum possible total contribution after dividing the array into exactly k non-empty contiguous segments.

Example

standard input	standard output
4 5 1 1 3 2 5 4 5 2 1 3 2 5 4 6 3 5 4 3 2 1 6 4 4 7 7 7 7	3 4 4 4

Note

In the first test case, there is only one segment: $[1, 3, 2, 5, 4]$. The contributing elements are 1, 3, and 5, so the contribution is 3.

In the second test case, an optimal division into segments is $[1, 3] \mid [2, 5, 4]$. The first segment contributes 2 and the second segment contributes 2, giving a total contribution of 4.

In the third test case, one optimal way to divide the array is $[5] \mid [4] \mid [3, 2, 1, 6]$. The contributions of the three segments are 1, 1, and 2, respectively, so the answer is 4.

Problem D. Skill Issue

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

Alice and Bob are preparing for a gaming tournament. The house has n devices, with m cables already directly connecting some pairs of them.

Alice plays on device a, Bob plays on device b, and the main router is device c. Every cable adds one unit of delay. To get the fastest connection, they want as few cables as possible between their device and the router.

They take turns adding one new cable between two devices. The only rule: the two devices must not already be connected by a sequence of cables — otherwise the cables would form a cycle, and a signal looping round and round would crash the network. The game ends when no more cables can be added.

Let $d(x, y)$ be the number of cables on the simple path between devices x and y at the end.

- If $d(a, c) < d(b, c)$, Alice gets the better connection and wins.
- If $d(b, c) < d(a, c)$, Bob gets the better connection and wins.
- If $d(a, c) = d(b, c)$, it is a tie.

Alice plugs in the first cable. Both gamers play perfectly: they will always make a move to secure a win if one exists. If winning is completely impossible, they will fiercely defend to force a tie rather than lose. What is the result of the game?

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains two integers n and m ($3 \le n \le 3 \cdot 10^5$, $0 \le m < n$) — the number of devices and the number of existing cables, respectively.

The second line of each test case contains three distinct integers a, b, and c ($1 \le a, b, c \le n$) — the device where Alice plays, the device where Bob plays, and the device where the main router is located, respectively.

Each of the next m lines contains two integers u and v ($1 \le u, v \le n$, $u \ne v$) — an existing cable connecting devices u and v. **It is guaranteed that the initial network contains no cycles.**

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

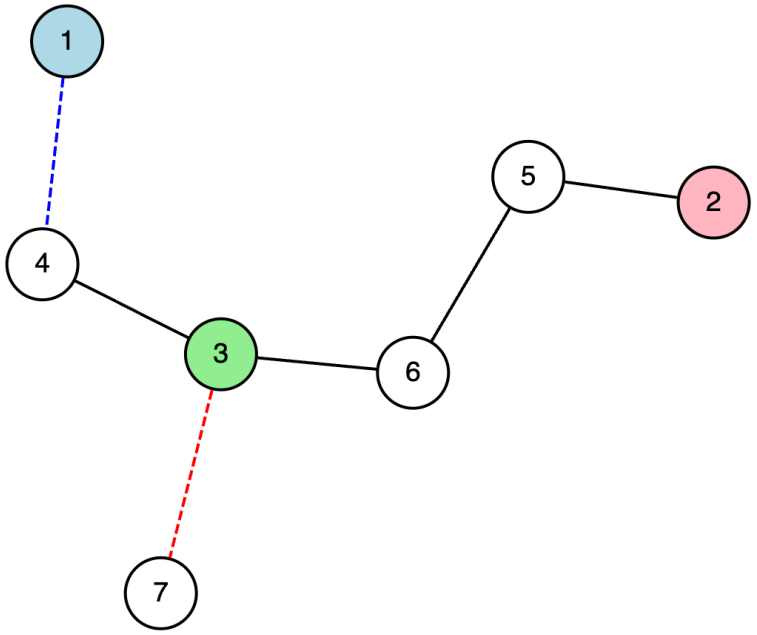
Output

For each test case, print a single line: `Alice` if Alice secures a faster connection, `Bob` if Bob does, or `Tie` if their connection speeds end up equal.

Example

standard input	standard output
2 7 4 1 2 3 4 3 2 5 5 6 6 3 7 3 1 2 3 1 4 2 5 3 6	Alice Tie

Note



For the first sample test case, Alice adds a cable between 1 and 4. Bob adds a cable between 3 and 7. The game ends. $d(1,3) = 2$ and $d(2,3) = 3$. Alice wins.

Problem E. Tangled

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given n distinct points on a plane. The coordinates of the i-th point are (x_i, y_i).

Your task is to calculate the number of pairs of indices (i, j) such that $1 \le i < j \le n$ and the line segment connecting points (x_i, y_i) and (x_j, y_j) intersects the line segment connecting points (y_i, x_i) and (y_j, x_j).

Two line segments are considered intersecting if they have at least one common point.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($2 \le n \le 3 \cdot 10^5$) — the number of points.

Each of the next n lines contains two integers x_i and y_i ($0 \le x_i, y_i \le 10^9$) — the coordinates of the i-th point.

It is guaranteed that all points in a test case are distinct.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print a single integer — the number of pairs of indices (i, j) satisfying the required condition.

Example

standard input	standard output
1 3 1 3 3 1 5 4	2

Note

In the first test case, all 3 pairs of indices are listed below:

- For pair $(1, 2)$, the segment connecting $(1, 3)$ and $(3, 1)$ is the same as the segment connecting $(3, 1)$ and $(1, 3)$, so they intersect.
- For pair $(1, 3)$, the segment connecting $(1, 3)$ and $(5, 4)$ intersects the segment connecting $(3, 1)$ and $(4, 5)$.
- For pair $(2, 3)$, the segment connecting $(3, 1)$ and $(5, 4)$ does not intersect the segment connecting $(1, 3)$ and $(4, 5)$.

The valid pairs are $(1, 2)$ and $(1, 3)$. Therefore, the answer is 2.

Problem F. Ebb and Flow

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given two non-negative integers n and m.

An array of non-negative integers $a_1, a_2, \ldots, a_n$ is *good* if for every index i ($1 \le i \le n$), the bitwise AND of the first i elements equals the bitwise OR of the last $n - i + 1$ elements. Formally, the array is good if

$$a_1 \,\&\, a_2 \,\&\, \cdots \,\&\, a_i = a_i \mid a_{i+1} \mid \cdots \mid a_n$$

holds for every $1 \le i \le n$.

Count the number of good arrays $a_1, a_2, \ldots, a_n$ such that $0 \le a_i \le m$ holds for every index i. Since the answer can be large, output it modulo 998244353.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The only line of each test case contains two integers n and m ($1 \le n \le 10^{18}$, $0 \le m \le 10^{18}$).

Output

For each test case, output a single integer — the number of good arrays modulo 998244353.

Example

standard input	standard output
3 2 2 3 4 5 2	5 19 11

Note

In the first test case, the good arrays are $[0, 0], [1, 0], [1, 1], [2, 0], [2, 2]$. For example, $[2, 0]$ is good because

- for $i = 1$, $a_1 = 2$ equals $a_1 \mid a_2 = 2$,
- for $i = 2$, $a_1 \,\&\, a_2 = 0$ equals $a_2 = 0$.

Problem G. An Odd Problem

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given three non-negative integers n, m, and k.

Find the maximum possible number of odd elements in an array of n integers $a_1, a_2, \ldots, a_n$ such that:

- $0 \le a_i \le m$ for every $1 \le i \le n$.
- The sum of the elements is exactly k, that is, $\sum_{i=1}^{n} a_i = k$.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first and only line of each test case contains three integers n, m, and k ($1 \le n, m \le 10^9$; $0 \le k \le n \cdot m$) — the size of the array, the maximum value of a single element, and the sum of all elements, respectively.

Output

For each test case, output a single integer — the maximum possible number of odd elements.

Example

standard input	standard output
4 5 3 5 3 3 4 5 2 8 4 4 16	5 2 2 0

Note

In the first test case, the array $[1, 1, 1, 1, 1]$ has sum 5, and all 5 of its elements are odd.

In the second test case, one optimal array is $[2, 1, 1]$, which has sum 4 and 2 odd elements.

In the third test case, one optimal array is $[2, 2, 2, 1, 1]$, which has sum 8 and 2 odd elements.

In the fourth test case, every element must equal 4 for the sum to be 16, giving $[4, 4, 4, 4]$, so none of the elements are odd and the answer is 0.

Problem H. Echoes of Erasure

Input file:	`standard input`
Output file:	`standard output`
Time limit:	3 seconds
Memory limit:	512 megabytes

You are given a string s of length n. Let $s[i..j]$ denote the substring $s_i s_{i+1} \dots s_j$.

For every pair of indices (i, j) with $1 \le i \le j \le n$, do the following:

- Take the substring $p = s[i..j]$ (of length $|p| = j - i + 1$);
- Erase p from s;
- Concatenate the two remaining parts into the string $w := s[1..i-1] + s[j+1..n]$, where $+$ denotes concatenation. Note that either part may be empty.

Let $f(i, j)$ be the number of occurrences of p as a substring in w. Here, an *occurrence* of p in w is a position x such that $w[x..(x+|p|-1)] = p$. Occurrences at different positions are counted separately, and they may overlap.

Compute $\sum_{1 \le i \le j \le n} f(i, j)$.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The only line of each test case contains a string s ($1 \le |s| \le 3 \cdot 10^5$) consisting of lowercase Latin letters.

It is guaranteed that the sum of $|s|$ over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, output a single integer — the value of $\sum_{1 \le i \le j \le n} f(i, j)$.

Example

standard input	standard output
4 abab aaaa aabaab z	6 15 20 0

Note

In the first test case, $s =$ `abab`, and the answer is 6 — of the ten pairs (i, j), six contribute one occurrence each and the rest contribute none:

- $(i, j) = (1, 1)$: erase `a`, leaving $w =$ `bab`, which contains `a` once.
- $(i, j) = (1, 2)$: erase `ab`, leaving $w =$ `ab`, which contains `ab` once.
- $(i, j) = (2, 2)$: erase `b`, leaving $w =$ `aab`, which contains `b` once.
- $(i, j) = (3, 3)$: erase `a`, leaving $w =$ `abb`, which contains `a` once.
- $(i, j) = (3, 4)$: erase `ab`, leaving $w =$ `ab`, which contains `ab` once.
- $(i, j) = (4, 4)$: erase `b`, leaving $w =$ `aba`, which contains `b` once.

Problem I. The Paintress

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You have n boards and n paint colors. The i-th board has length l_i, and there are c_j units of paint of color j.

The total length of all boards equals the total amount of paint: $\sum_{i=1}^{n} l_i = \sum_{j=1}^{n} c_j$.

You must use all the paint and fully cover every board. Formally, construct a matrix x of size $n \times n$, where $x_{i,j}$ is a non-negative integer equal to the number of units of board i painted with color j. It must satisfy $\sum_{j=1}^{n} x_{i,j} = l_i$ for every board i, and $\sum_{i=1}^{n} x_{i,j} = c_j$ for every color j.

A painting is called *simple* if the boards can be ordered as $p_1, p_2, \ldots, p_n$ so that, for every k from 1 to n, the first k boards $p_1, p_2, \ldots, p_k$ use at most k distinct colors in total — that is, the number of colors appearing on at least one of these k boards is at most k.

It can be shown that a *simple* painting always exists. You must output a *simple* painting together with such an order as proof that it is *simple*.

Input

The first line contains a single integer t ($1 \le t \le 100$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 1000$) — the number of boards and colors.

The second line contains n integers $l_1, l_2, \ldots, l_n$ ($1 \le l_i \le 10^9$) — the lengths of the boards.

The third line contains n integers $c_1, c_2, \ldots, c_n$ ($1 \le c_j \le 10^9$) — the amounts of each color of paint.

It is guaranteed that $\sum_{i=1}^{n} l_i = \sum_{j=1}^{n} c_j$, and that the sum of n over all test cases does not exceed 1000.

Output

For each test case, output $n + 1$ lines.

The first n lines describe the painting: the i-th of them contains n non-negative integers $x_{i,1}, x_{i,2}, \ldots, x_{i,n}$.

The last line contains n distinct integers $p_1, p_2, \ldots, p_n$ — a permutation of $1, 2, \ldots, n$ — such that for every k from 1 to n, boards $p_1, \ldots, p_k$ use at most k distinct colors in total.

If there are multiple valid answers, output any of them.

Example

standard input	standard output
1 3 5 2 3 2 5 3	0 2 3 2 0 0 0 3 0 3 1 2

Note

In the first test case, one valid painting is shown below, with the board lengths l_i in the right margin and the paint amounts c_j along the bottom:

	color 1	color 2	color 3	l_i
board 1	0	2	3	5
board 2	2	0	0	2
board 3	0	3	0	3
c_j	2	5	3	

Every row sum equals the board's length l_i and every column sum equals the paint amount c_j, so the painting is valid.

The order $p = [3, 1, 2]$ shows that the painting is *simple*. Add the boards one at a time in this order, keeping track of every color used so far:

- Take board 3 first: it uses only color 2, so 1 color has been used so far, and $1 \leq 1$.
- Add board 1: it uses colors 2 and 3, so the colors used so far are $\{2, 3\}$ — 2 colors, and $2 \leq 2$.
- Add board 2: it uses color 1, so the colors used so far are $\{1, 2, 3\}$ — 3 colors, and $3 \leq 3$.

At every step, the number of colors used so far is at most the number of boards taken so far, so the painting is *simple*.

Problem J. Know Your Place

Input file:	`standard input`
Output file:	`standard output`
Time limit:	1 second
Memory limit:	256 megabytes

You are given an array a of n positive integers.

You may perform the following operation any number of times:

- Choose an index i ($1 \le i \le |a|$) such that $a_i \le i$, where $|a|$ denotes the current length of the array, and delete the element a_i from the array; after the deletion, all elements to the right of position i shift one position to the left.

Determine whether it is possible to delete all elements of the array using these operations.

Input

The first line contains a single integer t ($1 \le t \le 10^4$) — the number of test cases.

The first line of each test case contains a single integer n ($1 \le n \le 3 \cdot 10^5$) — the length of the array.

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^9$) — the elements of the array.

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

For each test case, print `YES` if it is possible to delete all elements of the array, and `NO` otherwise.

Example

standard input	standard output
2 3 1 2 1 4 7 1 2 4	YES NO

Note

In the first test case,

- Select $i = 2$ as $a_2 \le 2$, delete a_2, the array becomes $[1, 1]$;
- Select $i = 2$ as $a_2 \le 2$, delete a_2, the array becomes $[1]$;
- Select $i = 1$ as $a_1 \le 1$, delete a_1, the array becomes empty.

Problem K. Relatively Speaking

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	256 megabytes

You are given a positive integer n.

For an integer k, let $f(k)$ be the number of integers x ($1 \le x \le n - k + 1$) such that $\gcd(x + i, n) = 1$ holds for every integer i with $0 \le i < k$. In other words, $f(k)$ counts the blocks of k consecutive integers $x, x + 1, \ldots, x + k - 1$ lying inside the range $[1, n]$ in which each of these k integers is coprime to n.

Compute $f(1) + f(2) + \cdots + f(n)$. Since the answer can be large, output it modulo 998244353.

Here, $\gcd(a, b)$ denotes the greatest common divisor of two integers a and b, and two integers are coprime if their greatest common divisor equals 1.

Input

The first line contains a single integer t ($1 \le t \le 100$) — the number of test cases.

The only line of each test case contains a single integer n ($1 \le n \le 10^{12}$).

Output

For each test case, output a single integer — the value of $f(1) + f(2) + \cdots + f(n)$ modulo 998244353.

Example

standard input	standard output
3 6 9 12	2 9 4

Note

In the third test case, $n = 12$. The integers in $[1, 12]$ coprime to 12 are $1, 5, 7, 11$, and no two of them are consecutive. Hence only single integers form valid blocks: $f(1) = 4$ and $f(k) = 0$ for $k \ge 2$. The answer is 4.